Practical No-5

Name: soham kulkarni Roll No: A - 58 BATCH - A - 4

Aim:Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries **Theory:**

---

1. Importance of Data Visualization & Role in EDA

Importance:

- Converts raw data into **visual format** (graphs, charts) for easy understanding
- Helps identify **patterns, trends, and relationships**
- Makes complex data **more accessible and interpretable** Supports
- **better decision-making**

Role in Exploratory Data Analysis (EDA):

- Detects **outliers and anomalies**
- Reveals **correlations between variables**
- Helps understand **data distribution**
- Guides further **data preprocessing and modeling**

Example: A scatter plot can reveal whether two variables are positively or negatively correlated.

---

2. Difference Between Matplotlib and Other Visualization Libraries

Feature	Matplotlib	Seaborn / Others
Level	Low-level	High-level
Flexibility	Very high	Moderate
Default Style	Basic	Attractive (built-in themes)
Ease of Use	More code required	Simpler syntax
Use Case	Full customization	Statistical visualization

Summary:

- **Matplotlib** → Highly customizable, foundational library
- **Seaborn** → Built on Matplotlib, easier for statistical plots and better aesthetics

---

3. Purpose of the `pyplot` Module in Matplotlib

- `pyplot` is a **collection of functions** used to create plots easily
- It provides a **MATLAB-like interface** Handles:
  - Creating figures
  - Plotting data
  - Adding labels, titles, legends Example:

```
import matplotlib.pyplot as plt
```

It allows quick plotting without managing low-level objects manually.

---

4. Process of Creating a Basic Line Plot

Steps:

1. Import library
2. Prepare data
3. Plot using `plt.plot()`
4. Add labels and title
5. Show the plot Example:

```
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

# Plot
plt.plot(x, y)

# Labels and title
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Basic Line Plot")

# Show plot
plt.show()
```

---

5. Importance of Customization & Options in Matplotlib

Why Customization Matters:

- Improves **clarity and readability**
- Makes visualizations **more informative**
- Helps highlight **important insights**
- Enhances **presentation quality**

Customization Options in Matplotlib:

- **Colors:** `color='red'`
- **Line styles:** `linestyle='--'`
- **Markers:** `marker='o'`
- **Titles & labels:** `plt.title()` `plt.xlabel()`
- **Legends:** `plt.legend()`
- **Grid:** `plt.grid()`
- **Figure size:** `plt.figure(figsize=(8,5))`
- **Axis limits:** `plt.xlim()`, `plt.ylim()`
- **Fonts & styles:** `plt.style.use('ggplot')`

Common Plots in Matplotlib & Seaborn

- **Scatter Plot** → Relationship between variables
- **Line Plot** → Trends over time
- **Bar Plot** → Comparison between categories
- **Histogram** → Data distribution
- **Box Plot**→ Spread and outliers
- **Pair Plot (Seaborn)**→ Relationships between multiple variables

---

Program

```
# Import libraries
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Load dataset (built-in dataset from seaborn)
df = sns.load_dataset('iris')

# Display first few rows
print(df.head())

# Set style
sns.set(style="whitegrid")

# Create a figure with multiple subplots
plt.figure(figsize=(15, 10))

# ------------------------------
# 1. Scatter Plot
# ------------------------------
plt.subplot(2, 3, 1)
plt.scatter(df['sepal_length'], df['petal_length'], color='blue')
plt.title("Scatter Plot")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

# ------------------------------
# 2. Line Plot
# ------------------------------
plt.subplot(2, 3, 2)
plt.plot(df['sepal_length'], label='Sepal Length', color='green')
plt.plot(df['petal_length'], label='Petal Length', color='red')
plt.title("Line Plot")
plt.xlabel("Index")
plt.ylabel("Values")
plt.legend()

# ------------------------------
# 3. Bar Plot
# ------------------------------
plt.subplot(2, 3, 3)
sns.barplot(x='species', y='sepal_length', data=df)
plt.title("Bar Plot")

# ------------------------------
# 4. Histogram
# ------------------------------
plt.subplot(2, 3, 4)
plt.hist(df['sepal_width'], bins=10, color='purple')
plt.title("Histogram")
plt.xlabel("Sepal Width")

# ------------------------------
# 5. Box Plot
# ------------------------------
plt.subplot(2, 3, 5)
sns.boxplot(x='species', y='petal_length', data=df)
plt.title("Box Plot")

# Adjust layout
plt.tight_layout()
plt.show()
```

```
# ------------------------------
# 6. Pair Plot (separate figure)
# ------------------------------
sns.pairplot(df, hue='species')
plt.show()
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```